

Trabalho Prático 1 — Algoritmo Genético

1. OBJETIVO

O objetivo deste trabalho é implementar e avaliar experimentalmente um **Algoritmo Genético (AG)** aplicado a problemas de otimização contínua. O trabalho busca compreender o funcionamento do algoritmo e analisar como diferentes escolhas relacionadas aos seus **operadores e parâmetros** influenciam seu comportamento.

2. PROBLEMAS DE OTIMIZAÇÃO

O Algoritmo Genético deverá ser aplicado às seguintes funções clássicas de otimização contínua:

DOMÍNIO DE BUSCA (para todas as funções): $x_i \in [-5, 5], i = 1, 2, \dots, n$

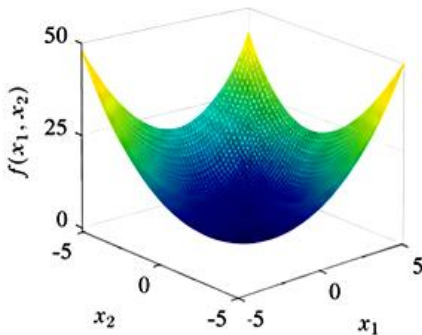
ÓTIMO GLOBAL CONHECIDO:

- Esférica: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (0, 0, \dots, 0)$
- Rastrigin: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (0, 0, \dots, 0)$
- Rosenbrock: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (1, 1, \dots, 1)$

Todos os problemas deverão ser tratados como problemas de **minimização**.

1) ESFÉRICA

$$f(\mathbf{x}) = \sum_{i=1}^n x_i^2$$



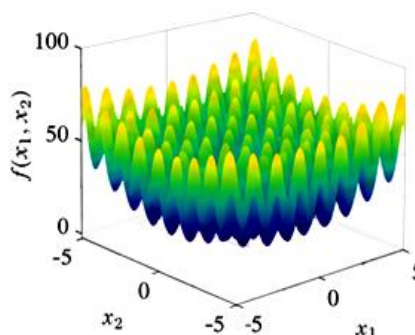
Domínio: $x_i \in [-5, 5], i = 1, 2, \dots, n$

Ótimo global: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (0, 0, \dots, 0)$

Características: unimodal, convexa, sem mínimos locais.

2) RASTRIGIN

$$f(\mathbf{x}) = 10n + \sum_{i=1}^n (x_i^2 - 10 \cos(2\pi x_i))$$



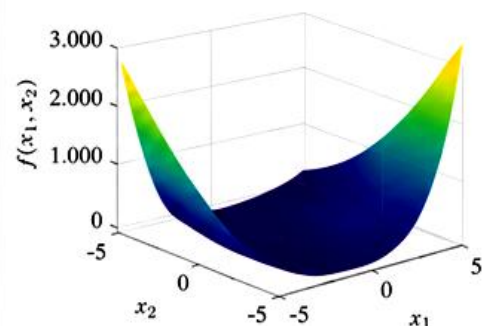
Domínio: $x_i \in [-5, 5], i = 1, 2, \dots, n$

Ótimo global: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (0, 0, \dots, 0)$

Características: multimodal, grande número de mínimos locais, altamente separável.

3) ROSENBROCK

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$$



Domínio: $x_i \in [-5, 5], i = 1, 2, \dots, n$

Ótimo global: $f(\mathbf{x}^*) = 0$ em $\mathbf{x}^* = (1, 1, \dots, 1)$

Características: unimodal, vale estreito e curvo, forte dependência entre variáveis.

Observação: n é a dimensão do problema (número de variáveis). As figuras acima ilustram as funções para $n = 2$.

3. IMPLEMENTAÇÃO DO ALGORITMO GENÉTICO

O Algoritmo Genético deverá ser implementado em **Python** e deverá contemplar explicitamente:

- representação do indivíduo;
- geração da população inicial;
- avaliação dos indivíduos;
- seleção dos pais;
- crossover;
- mutação;
- critério de parada.

As decisões de implementação são parte do trabalho. Portanto, os alunos deverão ser capazes de **explicar e justificar as escolhas realizadas**.

O código deverá permitir a configuração dos principais parâmetros do algoritmo, além dos parâmetros específicos dos operadores escolhidos.

N = tamanho da população

$p_c = P(\text{crossover})$

$p_m = P(\text{mutação})$

4. CONFIGURAÇÃO EXPERIMENTAL

O AG deverá ser avaliado para as dimensões: **$N = 2, 5$ e 10** , utilizando as quatro funções de teste apresentadas anteriormente.

Como o Algoritmo Genético é **estocástico**, uma única execução não é suficiente para avaliar seu desempenho. Cada configuração deverá ser executada **10 vezes**, utilizando diferentes sementes aleatórias. Os experimentos deverão utilizar a **mesma configuração básica do AG** para todas as funções, permitindo observar como um mesmo algoritmo se comporta diante de diferentes características do espaço de busca.

Orçamento computacional: O critério principal para limitar o esforço computacional será o **número de avaliações da função objetivo (NFE)**. Deve ser definido previamente o valor máximo de **20.000** de NFE, que será mantido nas comparações realizadas para uma mesma dimensão.

5. ANÁLISE DOS OPERADORES E PARÂMETROS

Além da execução da configuração básica, deverá ser realizado **um experimento específico para investigar o comportamento do Algoritmo Genético**. Este experimento poderá ser realizado utilizando apenas **uma das funções e uma das dimensões estudadas**.

Deverá ser escolhido **um** dos seguintes aspectos:

- método de seleção;
- operador ou probabilidade de crossover;
- operador ou probabilidade de mutação;
- tamanho da população.

Deverão ser comparadas pelo menos **duas configurações** para o aspecto escolhido.

Exemplo: torneio $\times$ roleta; ou duas probabilidades de mutação; ou dois tamanhos de população.

O objetivo deste experimento não é descobrir a “melhor configuração” do AG, mas **investigar e explicar como uma escolha influencia o comportamento da busca**. As configurações comparadas deverão utilizar o **mesmo número de avaliações da função objetivo (NFE)**.

6. RESULTADOS

Para cada cenário experimental, deverão ser apresentados:

- melhor solução encontrada;
- média das soluções finais;
- desvio padrão;
- número de avaliações da função objetivo;
- tempo médio de execução.

Deverão ser apresentados **gráficos de convergência**, mostrando a evolução da melhor solução encontrada em função do número de avaliações da função objetivo.

Algumas questões que podem orientar a análise:

- O aumento da dimensão tornou o problema mais difícil?
- O comportamento foi semelhante nas quatro funções?
- Houve grande variabilidade entre execuções independentes?
- O algoritmo convergiu rapidamente?
- Houve indícios de convergência prematura?
- A alteração estudada melhorou a qualidade das soluções?
- Houve alguma relação entre qualidade da solução e diversidade da população?
- Quais características das funções parecem ter influenciado o desempenho do AG?

7. ENTREGA E APRESENTAÇÃO

Deverão ser entregues, via Moodle:

1. **código-fonte** da implementação;
2. **relatório em PDF** apresentando a metodologia experimental, os resultados e sua análise.

O relatório deverá conter:

1. representação utilizada;
2. descrição dos operadores do AG;
3. parametrização;
4. configuração dos experimentos;
5. resultados e gráficos de convergência;
6. discussão dos resultados;

8. USO DE FERRAMENTAS DE INTELIGÊNCIA ARTIFICIAL

É permitida a utilização de ferramentas de **IA generativa** como apoio à implementação, documentação, depuração e análise do trabalho. Entretanto, o aluno é **responsável pelo conteúdo entregue**, independentemente das ferramentas utilizadas durante seu desenvolvimento.

Resultados ou decisões metodológicas que o aluno não consiga explicar ou justificar poderão não ser considerados na avaliação.

CÓDIGO PYTHON DAS FUNÇÕES

```
import numpy as np

# 1: Função Esférica
def sphere (x: np. ndarray ) -> float :
    return np. sum(x **2)

# 2: Função Rastrigin
def rastrigin (x: np. ndarray ) -> float :
    n = len(x)
    return 10 * n + np.sum(x**2 - 10 * np.cos (2 * np.pi * x))

# 3: Função Rosenbrock
def rosenbrock (x: np. ndarray ) -> float :
    return np. sum (100*( x [1:] - x [: -1])**2) **2 + (x[: -1] -1) **2)
```